

FDTD: solving 1+1D delay PDE in parallel

Y. L. Fang,

To be published in "CPC"

August 2018

Computational Science Initiative
Brookhaven National Laboratory

U.S. Department of Energy

USDOE Office of Science (SC), Advanced Scientific Computing Research (SC-21)

Notice: This manuscript has been authored by employees of Brookhaven Science Associates, LLC under Contract No. DE-SC0012704 with the U.S. Department of Energy. The publisher by accepting the manuscript for publication acknowledges that the United States Government retains a non-exclusive, paid-up, irrevocable, world-wide license to publish or reproduce the published form of this manuscript, or allow others to do so, for United States Government purposes.

This work was supported by the Laboratory Directed Research and Development Program of Brookhaven National Laboratory under U.S. Department of Energy Contract No. DE-SC0012704.

DISCLAIMER

This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, nor any of their contractors, subcontractors, or their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or any third party's use or the results of such use of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof or its contractors or subcontractors. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

FDTD: solving 1+1D delay PDE in parallel

Yao-Lung L. Fang^{a,b,c}

^a*Department of Physics, Duke University, P.O. Box 90305, Durham, North Carolina 27708-0305, USA*

^b*Computational Science Initiative, Brookhaven National Laboratory, Upton, NY 11973-5000, USA¹*

^c*National Synchrotron Light Source II, Brookhaven National Laboratory, Upton, NY 11973-5000, USA¹*

Abstract

We present a proof of concept for solving a 1+1D complex-valued, delay partial differential equation (PDE) that emerges in the study of waveguide quantum electrodynamics (QED) by adapting the finite-difference time-domain (FDTD) method. The delay term is spatially non-local, rendering conventional approaches such as the method of lines inapplicable. We show that by properly designing the grid and by supplying the (partial) exact solution as the boundary condition, the delay PDE can be numerically solved. In addition, we demonstrate that while the delay imposes strong data dependency, multi-thread parallelization can nevertheless be applied to such a problem. Our code provides a numerically exact solution to the time-dependent multi-photon scattering problem in waveguide QED.

Keywords: Waveguide QED, Delay PDE, FDTD, Non-Markovianity

PROGRAM SUMMARY

Program Title: FDTD: solving 1+1D delay PDE

Licensing provisions: MIT

Programming language: C (C99)

Supplementary material:

Journal reference of previous version:

Does the new version supersede the previous version?:

Reasons for the new version:

*Summary of revisions:**

Nature of problem(approx. 50-250 words): This program solves an unconventional 1+1D delay PDE that emerges in the study of waveguide quantum electrodynamics. The delay PDE is complex-valued and has a non-local delay term, and the solution to it provides the full dynamics of the system consisting of a few 1D photons and a two-level system in front of a mirror.

Solution method(approx. 50-250 words): The finite-difference time-domain (FDTD) method is adapted. Given the initial condition of the system, the corresponding boundary condition is generated, and then the FDTD solver marches through the entire spacetime grid. Multiple solvers are supported using either OpenMP (wavefront) or pthreads (swarm).

¹Present address.

Additional comments including Restrictions and Unusual features (approx. 50-250 words): 1. Depending on the input parameters the memory and disk usages of the program can be excessive, so the users should choose the parameters wisely (see main text). 2. The multi-thread support using OpenMP is turned on by default. See README for how to turn it off and switch to pthreads instead. 3. As a by-product, a numerical routine is provided for evaluating the incomplete Gamma function $\gamma(n, z)$ with nonzero positive integers $n \geq 1$ and complex-valued z .

1. Introduction

Waveguide quantum electrodynamics (QED) concerns the interaction between one-dimensional (1D) waveguide photons and local emitters (atoms, qubits, etc.) [1–5]. In the cases where a coherent feedback loop is formed due to the presence of, for example, multiple distant emitters or a perfect mirror terminating the waveguide, the propagation of photons needs to be taken into account rigorously if the time of flight between distant objects is non-negligible compared to the decay time of the qubits [6–11].

The study of time evolution (i.e., dynamics) in waveguide QED has drawn considerable attention [12–22], since it offers full information of light-matter interaction based on which more precise control of the system or detailed analysis of quantum non-Markovianity [23–25] can be performed. Insight into the non-equilibrium physics can also be obtained. While powerful numerical approaches, such as density matrix renormalization group (DMRG) [26–28] whose accuracy for solving 1D systems is undoubtedly demonstrated, have been applied recently to waveguide QED problems with feedback loops [11, 22], in the present paper and Ref. [29] we show that simply solving the *time-dependent* Schrödinger equation can provide a perhaps more natural and intuitive viewpoint on the scattering physics. However, in the presence of delay, the Schrödinger equation leads to a delay partial differential equation (PDE), solving which is a significant technical challenge to be addressed by this paper.

As a concrete example, we consider a two-level system (2LS) coupled to a semi-infinite waveguide, one end of which is terminated by a perfect mirror [8, 29]. Under the rotating-wave approximation, the number of excitations in a waveguide-QED system is conserved. Therefore, one may partition the full Hilbert space into different number sectors. While the dynamics in the one-excitation sector is described by a delay differential equation (DDE)¹, a one-variable ordinary differential equation that has a delay term [29–31] and can be solved straightforwardly [32], the multi-excitation sectors require nontrivial care. In particular, in the *two-excitation* sector there can be either two propagating photons in the waveguide (wavefunction denoted by χ hereafter), or one flying and the other absorbed by the 2LS (ψ); note that a 2LS can only hold one photon at a time. Therefore, starting from the Schrödinger equation and unfolding the half space, we arrive at a 1+1D delay PDE that describes the (complex-valued) time-dependent wavefunction, $\psi(x, t)$,

¹To be more consistent with respect to its PDE counterpart, one could use the acronym “delay ODE”, which stands for *delay ordinary differential equation*. However, we use DDE to follow the convention in the literature.

of the 2LS plus a photon at position x :

$$\begin{aligned}
\frac{\partial}{\partial t}\psi(x, t) = & -\frac{\partial}{\partial x}\psi(x, t) - \left(i\omega_0 + \frac{\Gamma}{2}\right)\psi(x, t) + \frac{\Gamma}{2}\psi(x - 2a, t - 2a)\theta(t - 2a) \\
& - \frac{\Gamma}{2}\left\{\left[\psi(-x - 2a, t - x - a) - \psi(-x, t - x - a)\right]\theta(x + a)\theta(t - x - a) \right. \\
& \left. + \left[\psi(2a - x, t - x + a) - \psi(-x, t - x + a)\right]\theta(x - a)\theta(t - x + a)\right\} \\
& + \sqrt{\Gamma}\left[\chi(x - t, -a - t, 0) - \chi(x - t, a - t, 0)\right],
\end{aligned} \tag{1}$$

where a is the atom-mirror separation, ω_0 (Γ) is the 2LS frequency (decay rate), $\theta(x)$ is the step function, $\chi(x_1, x_2, t)$ is the time-dependent two-photon wavefunction (one at position x_1 , another at x_2), and we set $\hbar = c = 1$. By solving Eq. (1) for ψ , the full dynamics of the system can be completely determined, since the two-photon wavefunction can be written in terms of ψ :

$$\begin{aligned}
\chi(x_1, x_2, t) = & \chi(x_1 - t, x_2 - t, 0) - \frac{\sqrt{\Gamma}}{2}\left[\psi(x_1 - x_2 - a, t - x_2 - a)\theta(x_2 + a)\theta(t - x_2 - a) \right. \\
& \left. - \psi(x_1 - x_2 + a, t - x_2 + a)\theta(x_2 - a)\theta(t - x_2 + a) + (x_2 \leftrightarrow x_1)\right].
\end{aligned} \tag{2}$$

Note that χ is symmetric under the exchange of x_1 and x_2 due to bosonic statistics. The detailed discussion of this problem and the derivation of above equations are reported in [29].

Before proceeding, we first reiterate that the focus of the present paper is on delay PDE, not DDE. As stated above, solving DDE subject to appropriate initial conditions, in particular linear DDE of the form $\dot{x}(t) = Ax(t - \tau) + Bx(t) + C$, is simple and in fact standard [32]. The solution usually consists of an infinite sum of piecewise functions whose interval is dictated by the delay τ . For solving more complicated cases, such as a nonlinear DDE, DDE with time-dependent coefficients, a system of DDEs, and possibly a combination of these, there are also sophisticated numerical routines provided in, e.g., Matlab and Mathematica. Second, while delay PDEs have emerged in some scientific and engineering contexts and been numerically studied using, for example, the method of lines [33], we emphasize that as far as we understand, those approaches cannot be directly applied to our problem, as the delay term in Eq. (1) lies in both x and t dimensions (i.e., it's a non-local delay), contrary to the common situation of delay PDE in which only one of the dimensions is delayed [33]. In other words, Eq. (1) cannot be converted to a system of ordinary differential equations (ODE) that is discretized in x , and then be solved by an ODE/DDE solver along t . Furthermore, to the best of our knowledge there is no general-purpose solvers for delay PDE. As a result, we adapt and implement the finite-difference time-domain (FDTD) method for solving the delay PDE (1) and demonstrate its validity in this paper.

Moreover, because of the feedback loop the system is highly “non-Markovian”, a jargon widely used in the community of open quantum systems [23–25] meaning the quantum system has a dependence on its past history. Since solving the wavefunction requires looking up the system’s memory (values of ψ solved at earlier times), as we will see this imposes stringent constraints on how the problem can be parallelized. In the paper, we present, implement, and benchmark two different multi-thread approaches, *swarm* and *wavefront*, to address this issue.

The main purpose of this work is therefore three-fold: (i) to numerically solve Eq. (1) using FDTD; (ii) to provide a proof of concept that FDTD works well for tackling complex-valued,

spatially non-local delay PDE and that certain degree of parallelism can be achieved; (iii) to present a numerically exact solution to the *time-dependent, multi-photon scattering* problem in waveguide QED.

2. Method and targeted problems

FDTD is widely used by engineers in antenna designs, computational electrodynamics, plasmonics, etc. Below we briefly discuss how FDTD works, and refer interested readers to Refs. [34, 35] for the details.

Like most of differential-equation solving methods, FDTD discretizes the spacetime, and different discretization schemes have their own advantages and disadvantages. For simplicity we choose a square lattice, and note in passing that in real FDTD applications the Yee (staggered) lattice is more common, as the conservation laws of EM fields are trivially hold on the Yee lattice. In the following we set the ratio of the spatial step Δ_x to the temporal step Δ_t equal to the speed of light $c = 1$ so that $\Delta_t = \Delta_x = \Delta$. In FDTD, $\Delta_x/\Delta_t \geq c$ is called the Courant condition and is hold when the algorithm is stable. See [Appendix A](#) for a simplified discussion on stability.

We next express every term in Eq. (1) using finite differences. We use the “leapfrog” prescription that has $O(\Delta^2)$ accuracy [36]:

$$\frac{\partial f(x + \Delta/2, t + \Delta/2)}{\partial t} \approx \frac{1}{\Delta} \left(\frac{f(x, t + \Delta) + f(x + \Delta, t + \Delta)}{2} - \frac{f(x, t) + f(x + \Delta, t)}{2} \right) \quad (3a)$$

$$\frac{\partial f(x + \Delta/2, t + \Delta/2)}{\partial x} \approx \frac{1}{\Delta} \left(\frac{f(x + \Delta, t + \Delta) + f(x + \Delta, t)}{2} - \frac{f(x, t) + f(x, t + \Delta)}{2} \right) \quad (3b)$$

$$f(x + \Delta/2, t + \Delta/2) \approx \frac{1}{4} \left(f(x + \Delta, t + \Delta) + f(x + \Delta, t) + f(x, t + \Delta) + f(x, t) \right) \quad (3c)$$

In the code we call Eq. (3) a “square”, because the value at the center of a square is approximated using the values at its four corners. These discretization rules apply to the terms in the first line of Eq. (1) so that given the values at three corners, the value at the top right corner of a square can be solved. As for those terms in the second and the third lines of Eq. (1), hereafter referred to as the source terms [29], we need a different representation:

$$f(x + \Delta/2, t) \approx \frac{1}{2} \left(f(x, t) + f(x + \Delta, t) \right), \quad (4)$$

which we call a “bar”. The reason for using bars over squares will become clear shortly.

Now we can put together squares and bars to discretize Eq. (1). For putting the problem on a computer, we also need to draw a “box” as we cannot walk through the entire spacetime indefinitely. As a result, we need to specify 4 parameters to define the box geometry: N_x , N_y , n_x , and Δ ; see Table 1 for the list of accepted input parameters. The corresponding layout is shown in Fig. 1. Note that (a) the initial condition is given on the line $t = 0$ (the purple stripe); (b) the boundary condition is given for not just one line, as needed for solving ordinary (space-local and time-local) PDEs, but for a wide area (the green region) because of the delay and source terms; (c) in order to reach $x \geq +a$ and to make the layout well-defined, we need n_x to be an integer multiple of 2 and $n_x \leq 2N_x$; (d) the step size Δ can be given *arbitrarily*, see [Appendix B](#).

It is illustrative to see how the FDTD solves Eq. (1). Fig. 2 shows a snapshot of the FDTD solver which marches in a space-then-time manner. Since the delay PDE is *chiral* (unidirectional) [29], for a given time t the solver (conceptually represented by the black cross) moves from the

Table 1: Summary of all input parameters accepted by our program.

Parameter	Description	Mandatory	Default
Nx	Defined such that total grid points (to be solved) along x is $2N_x + 1$ (so $x/\Delta \in [-N_x, N_x]$)	Yes	n/a
Ny	Total grid points along t (so $t/\Delta \in [0, N_y - 1]$)	Yes	n/a
nx	$n_x = 2a/\Delta$; n_x needs to be an integer multiple of 2 and $n_x \leq 2N_x$	Yes	n/a
Delta	Step size Δ (See also Appendix B)	Yes	n/a
k	Driving frequency k (in units of Δ^{-1})	Yes	0
k1, k2	Incident frequencies for photon #1 (#2) (in units of Δ^{-1})	No ²	0
w0	2LS frequency ω_0 (in units of Δ^{-1})	Yes	n/a
gamma	2LS decay rate Γ (in units of Δ^{-1})	Yes	n/a
init_cond	1: two-photon plane wave; 2: stimulated emission (one-photon exponential wavepacket); 3: two-photon exponential wavepacket	Yes	0
alpha	Wavepacket width α (in units of Γ)	No ³	0
alpha1, alpha2	Wavepacket width for photon #1 (#2) (in units of Γ)	No	0
identical_photons	whether or not the two incident photons are the same	No	1
save_chi	Output $\chi(a + \Delta, a + \Delta + \tau, t)$ as plain text	No ⁴	0
save_psi	Output $\psi(x, t)$ as plain text	No	0
save_psi_binary	Output $\psi(x, t)$ as binary	No	0
save_psi_square_integral	Output $\int dx \psi(x, t) ^2$ as plain text	No	0
measure_NM	See Appendix D	No ⁵	0
Tstep	Output the wavefunctions for every Tstep + 1 temporal steps	No	0
Nth	Number of solvers	No	1

² Needed when `init_cond=3` and `identical_photons=0`.

³ Needed when `init_cond=2`, and ineffective when `init_cond=1`.

⁴ These options cannot be simultaneously turned off (set to 0), or no output will be generated.

⁵ Requires `init_cond=2`.

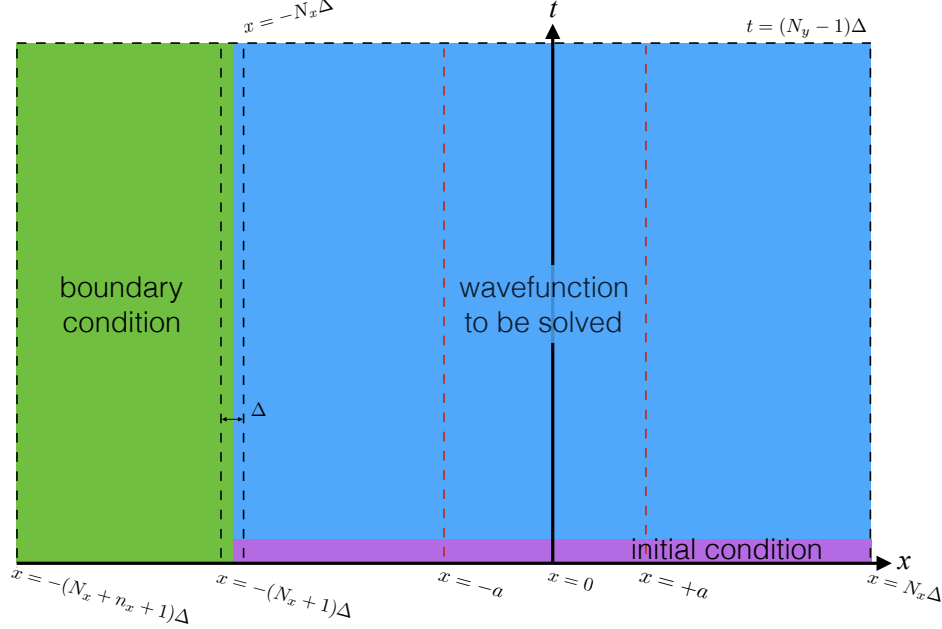


Figure 1: The schematic of the spacetime layout used in FDTD. Note that the lines $x = \pm a = \pm(n_x/2)\Delta$ locate in the blue region if $n_x \leq 2N_x$, and that $x = -a$ is at the center of the grid.

left edge of the box to the right, then advances one step Δ in time and repeats. Each term in Eq. (1) has a different color for easy identification, and we use all previously solved values to solve for the top-right corner of the blue square, which is circled in red. Note that there are four colors, each of which has only two points (the bars), because when we Taylor-expand at the black cross, those terms are expanded at the center between the two points.

Furthermore, from Fig. 2 one can appreciate the fact that in general the system is highly non-Markovian. In terms of programming, this brings in a heavy burden because, for performance reasons, one can no longer flush earlier values from memory to disk, as typically done in solving ordinary PDEs. Instead, one needs to keep all grid points in the memory, so the hardware capacity is an important factor; doing frequent I/O is not an efficient option. For example, as we are solving a complex wavefunction, depending on the grid size it can be very memory-intensive (each grid point stores a complex double number and thus takes 16 bytes) and space-intensive (the wavefunction is written to a plain txt when the calculation is done). Readers should use the program with caution.⁶

Currently the FDTD program can solve three classes of problems, and for each class the boundary condition (green in Fig. 1) is given by known analytical expressions:

- (a) *Two-photon plane wave* [29]: the incoming photons are described by continuous wave, $\chi(x_1, x_2, 0) = A^2 \exp[ik(x_1 + x_2)]\theta(-a - x_1)\theta(-a - x_2)$, and the initial condition for ψ is simply $\psi(x, 0) = 0$ (the 2LS is initially in its ground state). Therefore, we need to supply

⁶A quick estimation for memory usage is roughly $32N_xN_y/1024^3$ (in GB), and for disk usage divided by $T_{\text{step}} + 1$.

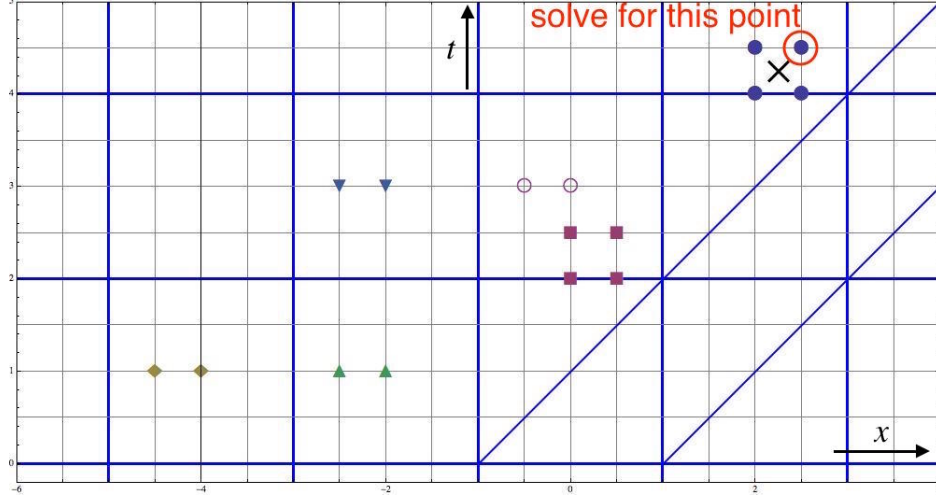


Figure 2: An example of calculating $\psi(x, t)$ for the circled point on the square lattice. The slant lines represent light cones extended from the coupling points at $x = \pm a$. n_x is chosen to be 4 for illustrative purposes, and the black cross denotes the Taylor-expansion point, referred to as the solver. The other colored points contribute to the point to be solved [each color corresponds to a term in Eq. (1)].

three more parameters that are physics-related: k , ω_0 , and Γ ; see Table 1. In $x < -a$, the solution to Eq. (1) is

$$\psi(x, t) = \sqrt{2}Ae^{ik(x-t)}e_0(t), \quad (5)$$

where $e_0(t)$ is the 2LS wavefunction, solved in the one-excitation sector assuming $e_0(0) = 0$ and $\phi(x, 0) = Ae^{ikx}\theta(-a - x)$,

$$e_0(t) = \frac{i\sqrt{\frac{\Gamma}{2}}Ae^{-ika}(e^{-ikt} - e^{-(i\omega_0 + \Gamma/2)t})}{p} - Ae^{-ika} \sum_{n=1}^{\infty} \frac{\left(\frac{\Gamma}{2}\right)^{n-1/2}}{n!} \left[(t - 2na)^n e^{-(i\omega_0 + \Gamma/2)(t - 2na)} + \frac{i^n(k - \omega_0)}{p^{n+1}} \gamma(n+1, -ip(t - 2na)) e^{-ik(t - 2na)} \right] \theta(t - 2na), \quad (6)$$

where $p = k - \omega_0 + i\Gamma/2$, and $\gamma(n, z)$ is the (lower) incomplete Gamma function [37]. We note that evaluating $\gamma(n, z)$ on the complex plane is in general a non-trivial task; see Appendix C. The above expressions are used to generate both the initial and boundary conditions. Finally, we note that $A = 1$ is set in the program for convenience.

(b) *Stimulated emission* [29]: a single-photon exponential wavepacket of the form⁷

$$\varphi(x) = i\sqrt{\alpha\Gamma}e^{ikx + \alpha\Gamma(x+a)/2}\theta(-x - a), \quad (7)$$

is sent in, with the 2LS initially excited, so $\psi(x, 0) = \varphi(x)$ and $\chi(x_1, x_2, 0) = 0$. In this case, one also needs to specify the wavepacket width α in the input file. In $x < -a$, the boundary

⁷The initial wavefront position is set at $x = -a$ to shorten the computation time and to maximize interference effects, otherwise we need to wait for the wavepacket to arrive, and by then the qubit may already decay.

condition is given by

$$\psi(x, t) = \varphi(x - t) \times e_1(t), \quad (8)$$

where $e_1(t)$ is the 2LS wavefunction solved in the one-excitation sector with initial conditions $e_1(0) = 1$ and $\phi(x, 0) = 0$,

$$e_1(t) = e^{-(i\omega_0 + \frac{\Gamma}{2})t} \sum_{n=0}^{\infty} \frac{1}{n!} \left[\frac{\Gamma}{2} e^{(i\omega_0 + \frac{\Gamma}{2})2a} (t - 2na) \right]^n \theta(t - 2na). \quad (9)$$

(c) *Two-photon exponential wavepacket* [38]: The corresponding initial conditions are $\psi(x, 0) = 0$ and

$$\chi(x_1, x_2, 0) = \frac{A}{\sqrt{2}} [\varphi_1(x_1)\varphi_2(x_2) + \varphi_1(x_2)\varphi_2(x_1)], \quad (10)$$

where A is the normalization constant such that $\iint dx_1 dx_2 |\chi(x_1, x_2, 0)|^2 = 1$ and $\varphi_i(x)$ is the i -th photonic wavepacket, assumed of the form Eq. (7) with incident frequency k_i and width α_i . The normalization constant can be chosen to be positive without loss of generality:

$$A = \sqrt{\frac{4(k_1 - k_2)^2 + (\alpha_1 + \alpha_2)^2 \Gamma^2}{4(k_1 - k_2)^2 + (\alpha_1 + \alpha_2)^2 \Gamma^2 + 4\alpha_1 \alpha_2 \Gamma^2}}. \quad (11)$$

Following our standard procedure, we first solve for $\psi(x < -a, t)$ and then plug it into the FDTD code to solve in the region $x > -a$. We find that the solution is given by

$$\psi(x < -a, t) = A [\varphi_1(x - t)e_0^{(2)}(t) + \varphi_2(x - t)e_0^{(1)}(t)], \quad (12)$$

where $e_0^{(j)}(t)$ is the qubit wavefunction in the one-excitation sector, solved subject to $e(0) = 0$ and $\phi(x, 0) = \varphi_i(x)$:

$$e(t) = \frac{\sqrt{\alpha\Gamma^2/2}(e^{-(i\omega_0 + \Gamma/2)t} - e^{-(ik + \alpha\Gamma/2)t})}{p} - i\sqrt{\alpha\Gamma} \sum_{n=1}^{\infty} \frac{\left(\frac{\Gamma}{2}\right)^{n-1/2}}{n!} [(t - 2na)^n e^{-(i\omega_0 + \Gamma/2)(t - 2na)} + \frac{i^n(k - \omega_0 - i\alpha\Gamma/2)}{p^{n+1}} \gamma(n + 1, -ip(t - 2na)) e^{-(ik + \alpha\Gamma/2)(t - 2na)}] \theta(t - 2na) \quad (13)$$

with $p = k - \omega_0 + i\Gamma/2(1 - \alpha)$. Note that when the two photons are identical, $\varphi_1 = \varphi_2 = \varphi$, the expression above reduces to the known form: $\psi(x < -a, t) = \sqrt{2}\varphi(x - t)e_0(t)$ [cf. Eq. (5)].

One could easily tweak the code to accept other kinds of initial conditions, but the strategy for solving Eq. (1) remains the same for all possible scenarios. It is important to note that due to the nature of the delay PDE (1), one cannot solve it numerically without knowing its analytical solutions in $x < -a$. But one does not need more than that either — the constraint $n_x \leq 2N_x$ guarantees that the knowledge in $x < -a$ is sufficient, as it makes the line $x = -a$ lie outside of the green region in Fig. 1 so that the boundary condition is completely determined.

Finally, to calculate various non-Markovian measures using the solution of ψ , instead of post-processing it is much easier to do most of the work *in situ*. To construct the geometric measure [39], two functions $\lambda(t)$ and $\mu(t)$ can be calculated [29].⁸ The detail of the implementation is presented in Appendix D.

⁸In Ref. [29] $\mu(t)$ is called $c(t)$ and $\lambda(t)$ is called $\Delta(t)$, respectively. The author regrets the inconvenience.

3. Parallelization

As discussed above, due to the non-local delay term there is a strong data dependency — the present solution depends on its past history. However, even with such a severely constrained problem, interestingly we find that the delay PDE (1) can still be solved in parallel, by which we mean that the simultaneous presence of multiple FDTD solvers in the spacetime is allowed. In this paper, we propose and implement two kinds of parallel approaches, referred to as the “swarm” and “wavefront” approaches, respectively, that respect the constraints imposed by the delay PDE (1).

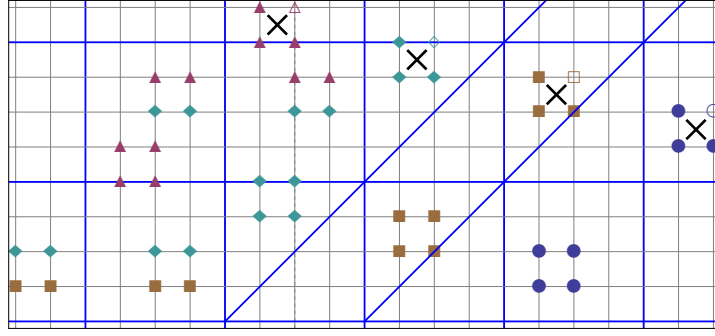


Figure 3: A snapshot of multiple marching FDTD solvers in the swarm approach. Each color and shape corresponds to points read and written by a certain solver (thread). The points to be solved by each solver are empty, while known points are filled. Note that each solver is one step above and at least n_x steps behind its predecessor (here $n_x = 4$).

For the swarm approach (see Fig. 3), when solving Eq. (1) from the bottom up, each solver must be one temporal step above and at least n_x spatial steps behind its next neighbor (called predecessor hereafter), or the execution order would not be preserved correctly; that is, one solver may read the value at certain point which is yet to be solved by another solver. It turns out that it is beneficial to have a “cyclic lag” relation among the solvers: denoting the total number of solvers as N_{th} , which can be set in the input file, then #2 is lagged behind #1, #3 behind #2, etc., and finally #1 can be set behind $\#N_{th}$ once it reaches the grid boundary. We find that such a wrap-around can increase performance and is easy for coding.

In the current implementation, we utilize the “wait-and-signal” mechanism provided by POSIX Threads (pthreads). Each solver is marched by an independent thread and has a lock for its position and a condition variable for waiting for its predecessor. Before attempting to solve at a certain point, it must look up its predecessor’s position and see whether the constraint is satisfied or not. If not, then it must wait for the signal sent by its predecessor before proceeding. Using pthreads’ locks not only allows a solver to communicate with and wait for its predecessor, but also preserves cache coherence, which is important since very often a solver needs to access a value immediately after it’s solved by another solver.

For the wavefront approach (see Fig. 4), we circumvent the requirement of separating n_x steps in space by partitioning the spacetime into three regions: $x \leq -a$, $-a < x \leq a$, and $x > a$, which are solved in turn. In the first and last regions, Eq. (1) can be solved by propagating the wavefront along the diagonal, and points on the same wavefront (along which the spatial lag between adjacent solvers is only one step) can be solved in parallel. In the middle region $-a < x \leq a$, which is n_x -step wide, due to causality we use only one solver (as if no parallelization

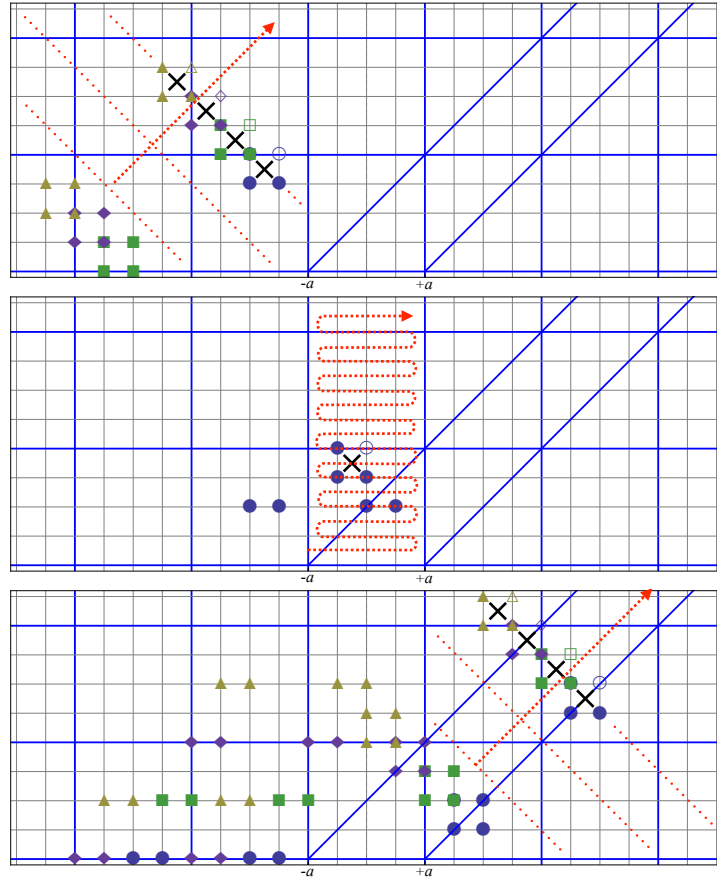


Figure 4: Schematic of the wavefront approach. We first solve in the region $x \leq -a$ (top), then $-a < x \leq a$ (middle), and finally $x > a$ (bottom). In the first and last regions, points on the same wavefront (represented by dotted lines) can be solved in parallel. In the middle region, due to causality only one solver is permitted to enter and move row by row. The solver(s) are propagated along the dashed lines in all regions. The colors and shapes of all points have the same meaning as in Fig. 3.

exists). Physically, it is clear [29] that in $x \leq -a$ ($x > a$) we only have right- (left-) going photons, and in $-a < x \leq a$ they are connected by the mirror.

We implement the wavefront approach using OpenMP, since within each wavefront it is simply an *embarrassingly parallel* problem; that is, there is no data dependency and thus no need to lock or signal. Moreover, because most of the time a wavefront typically contains at least hundreds of points, the synchronization penalty (due to the implicit barrier in the OpenMP constructs) is small compared to the gain from parallelization, and the barrier in turn helps preserve the cache coherence.

In the end of the next section, we present a strong-scaling measurement for a typical problem size required by the physics [29, 38, 40]. It will be shown that the wavefront approach outperforms the swarm approach.

4. Validation

User instruction for the program is given in the README.md file distributed along with the code. We have tested our program on both Linux and Mac OS X. There are at least three possible tests to pass for establishing the validity of this code: 1. ψ in $x \leq -a$, where the full, exact solution can be obtained; 2. ψ in $-a < x \leq a$ because we can solve for the first few triangular tiles analytically [29]; 3. the two-photon wavefunction χ , because the steady-state result is known from scattering theory [8].

For the first test, in Fig. 5 we show several snapshots of the absolute and relative errors of the real part of ψ as a function of steps in the x -direction (and $x \leq -a$ in all plots). We note that in order to reach the steady state, usually the termination time needs to satisfy $t \gg 1/\Gamma$. In addition, note two common features in these plots: (a) the error is zero in the initial steps because it's where the boundary condition resides; (b) errors are well-controlled in the sense that they are bounded in an oscillating envelope, so the program behaves correctly in $x \leq -a$.

For the second test, we generated the analytical expressions of ψ in the first four tiles (the fifth one took too much time to compute) in Mathematica [29] and then compared the values on the FDTD grid points. The numerical result agreed well too (not shown).

Finally, to compare with the scattering theory we can construct the two-photon wavefunction $\chi(x_1, x_2, t)$ using the calculated ψ according to the formal solution of χ , Eq. (2), where the coordinates x_1 and x_2 should be chosen as $x_{1,2} \geq +a$ for capturing the outgoing fields. In the program, χ is calculated by setting $x_1 = a + \Delta$ and $x_2 = a + \Delta + \tau$, with $\tau = x_2 - x_1$ being the separation of the two detectors.⁹ The results are shown in Figs. 6-8. The agreement is quite well, even in the non-Markovian regime in which $\Gamma a/c \gtrsim 1/2$ (Fig. 8). Because evaluating Eq. (2) requires the full history of ψ , it is clear the program is valid for all regions in the spacetime. More results generated by our FDTD program are discussed in Refs. [29, 38, 40].

We next comment briefly on the performance of multi-thread support. In Fig. 9 we report the elapsed time as a function of the number of threads (specified by Nth) for solving Eq. (1) using both the swarm and wavefront approaches for the same input parameters, which are chosen to roughly match those used in Refs. [29, 38, 40]. In order to have a fair comparison, note that when single thread is used, the threading libraries would impose unnecessary overhead, which is enormous in particular for the wavefront approach since after every step a barrier synchronization is enforced. Therefore, we also report in Fig. 9 the measurement without using any

⁹We note that by definition the two-photon correlation function is given by $g_2(\tau) = |\chi(x_d, x_d + \tau, t \rightarrow \infty)|^2$ with $x_d \geq +a$ ($x_d + \tau \geq +a$) being the position of the first (second) detector.

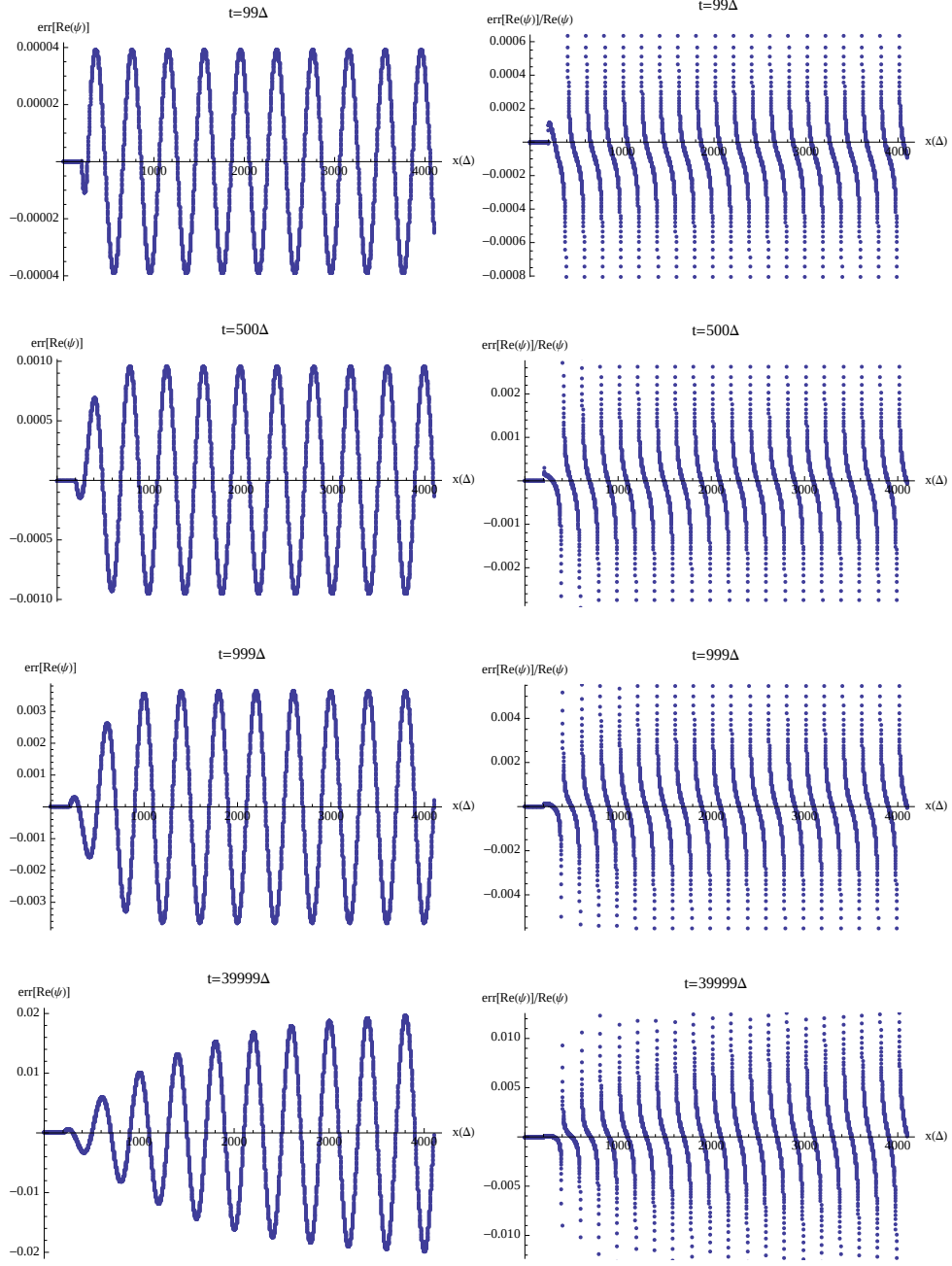


Figure 5: Absolute (left) and relative (right) errors for $\text{Re}(\psi)$ as a function of spatial steps (in units of Δ) at $t/\Delta = \{99, 500, 999, 39999\}$ (from top to bottom). The last step corresponds to $x = -a$. Input parameters: $n_x = 200$, $N_x = 4000$, $N_y = 4 \times 10^4$, $\Delta = 10^{-2}$, $k = \omega_0 = \pi/2$, $\Gamma = \pi/40$.

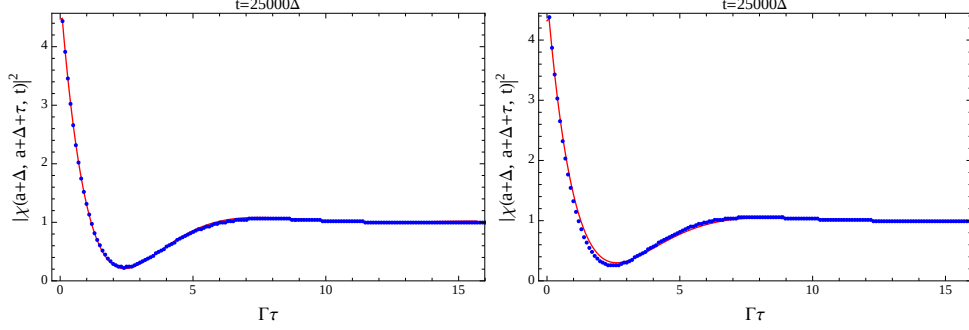


Figure 6: Comparison of calculated two-photon wavefunction $|\chi|^2$ (red curves) as a function of photon separation τ in the long-time limit for $k_0 a = \pi/4$ and (left) $k = \omega_0$ (right) $k = \omega_0 - \Gamma$. The blue dots are from the numerically exact scattering theory [8]. Input parameters: $n_x = 50, N_x = 10^4, N_y = 2.5 \times 10^4, \Delta = 10^{-2}, \omega_0 = 3.1415926536, \Gamma = 0.1570796327$.

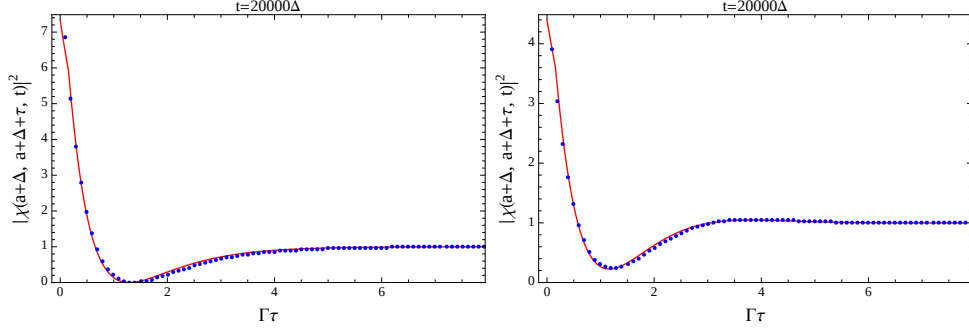


Figure 7: Comparison of calculated two-photon wavefunction $|\chi|^2$ (red curves) as a function of photon separation τ in the long-time limit for $k_0 a = \pi/2$ and (left) $k = \omega_0$ (right) $k = \omega_0 - \Gamma$. The blue dots are from the numerically exact scattering theory [8]. Input parameters: $n_x = 200, N_x = 10^4, N_y = 2 \times 10^4, \Delta = 10^{-2}, \omega_0 = 1.5707963268, \Gamma = 0.0785398163$.

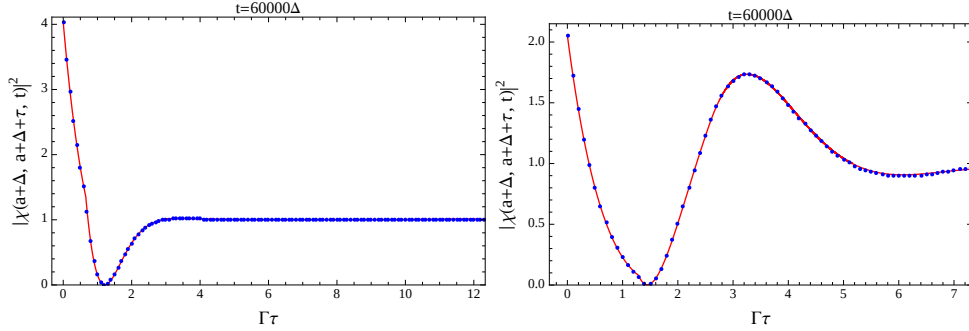


Figure 8: Comparison of calculated two-photon wavefunction $|\chi|^2$ (red curves) as a function of photon separation τ in the long-time limit for $k = \omega_0$ and (left) $k_0 a = 10.5\pi$ (right) $k_0 a = 20.5\pi$. The blue dots are from the numerically exact scattering theory [8]. Input parameters: $N_x = 6.48 \times 10^4, N_y = 6 \times 10^4, \Delta = \pi/12000, k = \omega_0 = 100, \Gamma = 1$.

threading mechanism (light blue circles), compared with which the speedup is close to 2x using the wavefront approach with 16 threads. We find that the wavefront approach performs much

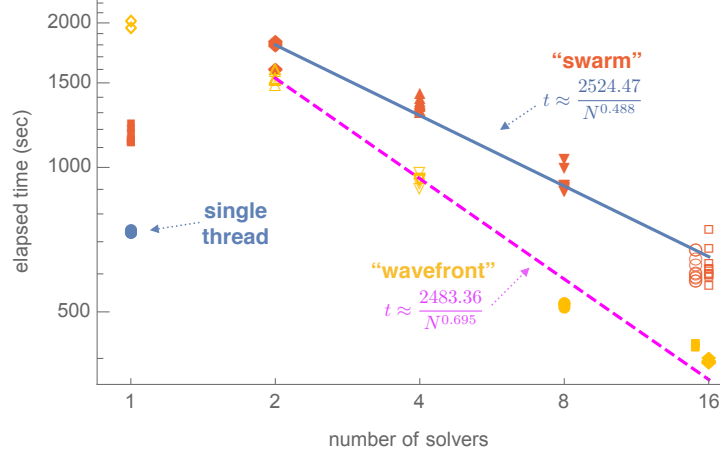


Figure 9: Log-log plot of measured elapsed time as a function of the number of solvers (threads) set by `Nth` using the swarm (orange) and the wavefront (yellow) approaches. The test is run 10 times for each case. For single thread, the measurements without threading overhead (light blue circles) are also shown. The straight lines are fitted by all data points with `Nth = 1` excluded. Input parameters: $N_x = 64800$, $N_y = 60000$, $n_x = 960$, $\Delta = 2.618 \times 10^{-4}$, `init_cond = 2`, $k = \omega_0 = 100$, $\Gamma = 5$, and $\alpha = 0.5$. Test environment: Intel Xeon CPU E5-2670 2.6 GHz with 16 physical cores, 128 GB physical memory, and gcc 7.2.0.

better than its swarm counterpart: it has a better scaling and is already faster than single thread with only 8 threads. However, both approaches do provide shorter runtime than that of the serial version. Finally, we note in passing that the test is performed on a machine with hyperthreading turned off, so we are unable to test with more threads, but we expect to see a consistent scaling behavior when running on machines with > 16 physical or logical cores.

5. Conclusion

The program provides a numerically exact, *time-dependent* solution to the problem of a single 2LS placed in front of a mirror scattered by either a one- or two-photon initial state, so for waveguide-QED researchers this program can be very useful for solving the three classes of problems as presented above. Arbitrary wavepackets or other forms of initial conditions can be incorporated into the code with minor modification.

More flexibility, such as a giant atom that has arbitrary number of distant legs coupled to the waveguide [20, 41, 42], can be gained by re-implementing this program in an OO language (C++, Python, etc). In fact, the code has been written in the OO-style by (i) compacting all relevant fields into a C struct (roughly equivalent to a C++/Python class) named `grid`, (ii) passing “this” pointer to a `grid` instance when calling functions, as if they were member functions of the `grid` class, and (iii) following RAII. As a result, rewriting in C++, for example, should be of minimal work. Of course, the `grid` layout must be carefully designed in such a scenario to minimize memory usage. As for the case of an infinite waveguide, we believe the best design is to use MPI for communication between the left- and right-moving solvers, which we leave for future work.

Finally, this program serves as a proof of concept for mathematicians, scientists and engineers who are interested in solving complicated delay PDE using parallel FDTD. Important issues such as general proof of stability, more efficient multi-thread implementation, robust optimization over

any 1+1D delay PDE, etc., remain challenging. For development discussions, please either open an issue on GitHub¹⁰ or contact the authors. Users and developers who are benefited from this program are strongly encouraged to cite both this paper as well as Ref. [29].

6. Acknowledgments

We acknowledge financial support from U.S. NSF (Grant No. PHY-14-04125) and the Brookhaven National Laboratory's Laboratory Directed Research and Development project #17-029, and fruitful discussion with Harold U. Baranger, Francesco Ciccarello and Meifeng Lin. We also thank Weiguo Yin for proofreading. Part of the tests used resources of the Center for Functional Nanomaterials, which is a U.S. DOE Office of Science User Facility, at Brookhaven National Laboratory under Contract No. DE-SC0012704.

Appendix A. Stability analysis

Following Ref. [36], we can perform a simple von Neumann stability analysis to check that our regularization scheme does not lead to amplitude divergence. First, let us consider the simplest ordinary PDE $\partial_x f + \partial_t f + Wf(x, t) = 0$ [compared with Eq. (1), $W = (i\omega_0 + \frac{\Gamma}{2})$]. If we write $\psi(m\Delta, n\Delta) = \xi^n e^{ik(m\Delta)}$, where ξ is called the amplification factor, then we have

$$\xi(k) = \frac{\left(\frac{1}{\Delta} - \frac{W}{4}\right) - \frac{W}{4}e^{ik\Delta}}{\left(\frac{1}{\Delta} + \frac{W}{4}\right) + \frac{W}{4}e^{-ik\Delta}}e^{-ik\Delta}. \quad (\text{A.1})$$

It can be shown that $|\xi(k)| \leq 1$ as long as $\text{Re}(W) > 0$, so our approach for solving this simple PDE is stable.

For Eq. (1) in $x < -a$, we can again insert the ansatz and obtain (assuming $\chi = 0$ for simplicity)

$$\begin{aligned} \left(\frac{1}{\Delta} + \frac{W}{4}\right)\xi^n e^{ikm\Delta} &= \left(\frac{1}{\Delta} - \frac{W}{4}\right)\xi^{n-1} e^{ik(m-1)\Delta} - \frac{W}{4}(\xi^{n-1} e^{ikm\Delta} + \xi^n e^{ik(m-1)\Delta}) \\ &\quad + \frac{\Gamma}{8}\xi^{n-n_x-1} e^{ik(m-n_x-1)\Delta} (1 + e^{ik\Delta} + \xi + \xi e^{ik\Delta}). \end{aligned} \quad (\text{A.2})$$

Note that the parameter n_x enters because of the delay term. Now this is a polynomial in ξ of order n_x , so there is no general solution for its roots. But empirically (i.e. by solving the equation graphically) we still find that $|\xi(k)| \leq 1$; that is, our algorithm is stable in $x < -a$.

A more general stability analysis for Eq. (1) is arduous. However, we hope the above reasoning together with the validations presented above are enough to convince the interested users that our FDTD program is stable.

¹⁰<https://github.com/leofang/FDTD>

Appendix B. Conversions between physical quantities and simulation parameters

In the scattering problem, the three important parameters are $\mathcal{K}$ ($= k/\Gamma$), $\mathcal{W}$ ($= \omega_0/\Gamma$) and n ($= k_0 a/\pi$). To satisfy the rotating-wave approximation $\mathcal{W} \gg 1$ is required, but how large $\mathcal{W}$ is should not matter. Once they are determined, the physics is determined. Here we describe how to express physical quantities in terms of Δ , n_x , $\mathcal{K}$, $\mathcal{W}$ and n :

$$k = \frac{2n\pi\mathcal{K}}{n_x\mathcal{W}} \times \frac{1}{\Delta} \quad (\text{B.1})$$

$$\omega_0 = \frac{2n\pi}{n_x} \times \frac{1}{\Delta} \quad (\text{B.2})$$

$$\Gamma = \frac{2n\pi}{n_x\mathcal{W}} \times \frac{1}{\Delta} \quad (\text{B.3})$$

$$\lambda_0 = \frac{n_x}{n} \times \Delta \quad (\text{B.4})$$

A Python script is used to prepare the input parameters using the above relations.

We remark the role of Δ , which represents both length and time in dimensional analysis (recall $c = 1$). Thus, we have a degree of freedom (dof) to choose its value without affecting the physics, as if we were changing the length unit. We emphasize this notion because there is in fact a deeper reason for having this dof: Eq. (1) is a wave equation, and thus *scale invariant* [43]. It is well-known that FDTD is quite suitable for scale-invariant problems, and the step size Δ is just a conceptual quantity for proper discretization and is irrelevant of the physics. What matters is the dimensionless quantities such as $(\omega_0\Delta)$ and $(\Gamma\Delta)$, not Δ itself.

The current implementation is not yet strictly scale invariant — we still explicitly keep Δ as a mandatory parameter, whose value does not affect the result (providing rounding errors are negligible), but it is not necessary if the code is implemented in a different way. What really controls the (relative) step size, and therefore the error, is the ratio of n_x/n ; see Eq. (B.4). As a result, we need $n_x \gg n$ to have many steps per wavelength. Empirically we find $\Delta/\lambda_0 \leq 1/100$ is desirable.

Appendix C. Evaluating incomplete Gamma functions on the complex plane

In the present work, the incomplete Gamma function $\gamma(n, z)$ naturally emerges from our equations. While there are several open-sourced math libraries such as GSL and Boost that implement the real-valued version, evaluating $\gamma(n, z)$ for complex-valued argument z is a very tricky task and to our knowledge no open-sourced code provides such a routine. Commercial softwares like Mathematica does provide one for arbitrary arguments, but its efficiency is nonideal for simulation purposes. As a by-product, therefore, we provide an implementation of the incomplete Gamma function for nonzero positive integers $n = 1, 2, \dots$, and complex-valued z .

Specifically, we implement the normalized lower incomplete Gamma function [37]

$$P(n, z) = \frac{\gamma(n, z)}{\Gamma(n)} = \frac{\gamma(n, z)}{(n-1)!}. \quad (\text{C.1})$$

Other members in the incomplete-Gamma family can be easily obtained if $P(n, z)$ is known. The most challenging part is when z is on the negative real axis. Fortunately, this problem is recently tackled in Ref. [44], and we incorporate their findings into our implementation, which works

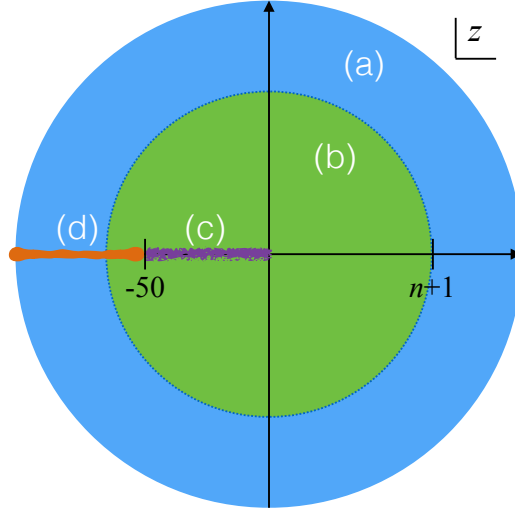


Figure C.10: Evaluating $P(n, z)$ for complex-valued z . Depending on $|z|$, we use different formulae, labeled from (a) to (d), to achieve fast and accurate convergence. Note that (c) and (d) are employed when $\text{Re}(z) < 0$ and $|\text{Im}(z)| \lesssim 10^{-16}$.

well even when z has a tiny, nonzero imaginary part. Our implementation is summarized in Fig. C.10. We evaluate $P(n, z)$ according to four different expressions for z in different regions on the complex plane:

- (a) Continuous fraction; see Eq. (6.2.7) in Ref. [36]:

$$Q(n, z) = 1 - P(n, z) = \frac{e^{-z} z^n}{\Gamma(n)} \left(\frac{1}{z+1-n} - \frac{1 \cdot (1-n)}{z+3-n} \frac{2 \cdot (2-n)}{z+5-n} \cdots \right); \quad (\text{C.2})$$

- (b) Series expansion; see Eq. (6.2.5) in Ref. [36]:

$$P(n, z) = e^{-z} z^n \sum_{i=0}^{\infty} \frac{z^i}{\Gamma(n+i+1)}; \quad (\text{C.3})$$

- (c) Series expansion for γ^* ; see Eq. (6) in Ref. [44]:

$$P(n, z) = z^n \gamma^*(n, z) = \frac{z^n}{\Gamma(n)} \sum_{i=0}^{\infty} \frac{(-z)^i}{i!(i+n)}. \quad (\text{C.4})$$

- (d) Poincaré-type expansion; see Eq. (29) in Ref. [44]:

$$P(n, -z) = (-z)^n \gamma^*(n, -z) \sim \frac{(-1)^n e^z}{\Gamma(n)} \sum_{i=0}^{\infty} (1-n)_i z^{n-i-1}; \quad (\text{C.5})$$

We note that combining (a) and (b) gives the standard approach to real-valued z [36], and that for very large $|z|$ the computation may not converge [44], but the convergence range is large enough for our purposes.

Appendix D. Non-Markovian measures

In this Appendix we illustrate the computation of the non-Markovian (NM) geometric measure [39] quantifying the single-photon scattering process in this system. Specifically, we consider initially the waveguide has a single-photon exponential wavepacket of the form Eq. (7), and calculate two functions, $\mu(t)$ and $\lambda(t)$,⁸ for constructing the geometric measure. The detailed discussion is reported elsewhere [29], and here we simply quote the results. The computation can be performed by setting an arbitrarily positive `alpha`, `init_cond=2`, and `measure_NM=1` in the input file.

Denoting $\phi(x, t)$ as the photon wavefunction in the one-excitation sector, we can define a complex function $\mu(t)$ as

$$\mu(t) \equiv \int_{-\infty}^{\infty} dx \phi^*(x, t) \psi(x, t). \quad (\text{D.1})$$

Similarly, we define a real function $\lambda(t)$ as

$$\lambda(t) = \int_{-\infty}^{\infty} dx |\psi(x, t)|^2 - |e_0(t)|^2, \quad (\text{D.2})$$

where $e_0(t)$ is given by Eq. (13). After calculating $\psi(x, t)$, the program will compute $\mu(t)$, $\lambda(t)$, $e_0(t)$ and $e_1(t)$ [Eq. (9)], and output the results as plain text. The geometric measure is defined as [39]

$$\mathcal{N}_{\text{geo}} = \int_{\frac{d|\det M_t|}{dt} > 0} \frac{d|\det M_t|}{dt} dt, \quad (\text{D.3})$$

and elsewhere [29] we show that for our system $\det M_t = |\mu(t)|^2 \lambda(t)$. Therefore, once the two functions $\lambda(t)$ and $\mu(t)$ are known, the geometric measure can be calculated easily.¹¹

For users interested in computing the two functions themselves, we note that the spatial integrals in Eqs. (D.1) and (D.2) need some care, which is explained in the rest of this section. (In our code they are computed according to the trapezoidal rule.) One of the issues for numerically computing these integrals is that the amount of data generated by the FDTD program can be huge, as previously explained. It is certainly inconvenient to write data into a file and subsequently load them into the memory for post processing.

We suggest a manageable way to reduce the disk and memory usages. First, note that the analytical solution in $x < -a$ is known, so do not write data in this region into files. The resulting data layout is shown in Fig. D.11. Doing this will cut half of the disk usage (compare Fig. D.11 with Fig. 1). Next, depending on the system time scale, one can selectively load data for every, say, 100 steps (blue stripes in Fig. D.11; set `Tstep=99`). This will significantly reduce the memory usage. One can keep loading data into the memory until reaching $t = (N_y - 1)\Delta$, the max simulation time in FDTD, but for the purpose of calculating the integrals, one may only use data up to $t = T_{\text{max}}\Delta$, where

$$T_{\text{max}} = \min(N_y - 1, N_x - n_x/2). \quad (\text{D.4})$$

¹¹ $\lambda(t)$ and $\mu(t)$ can also be used to construct other NM measures; detail in progress will be reported elsewhere.

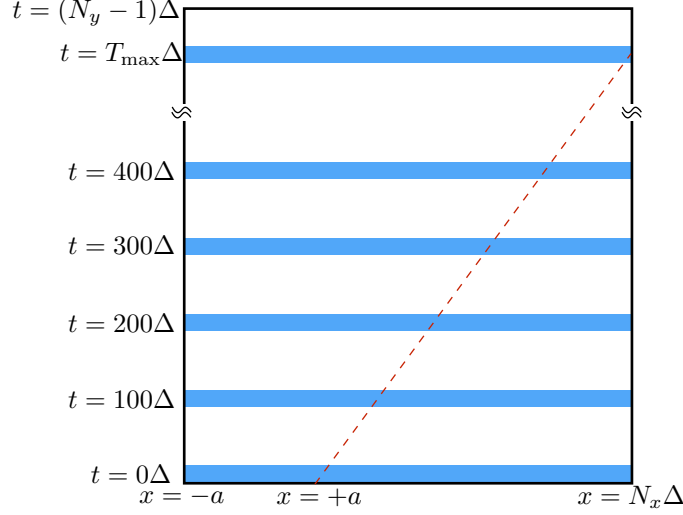


Figure D.11: Spacetime layout of the FDTD output files. Information for $x < -a$ is not written to the disk in order to reduce the file size. During post processing, one can load data for every 100 steps into memory (blue stripes). The red dashed line represents the light cone extended from the second 2LS at $x = +a$, and the time at which it intersects with the box boundary is denoted $t = T_{\max}\Delta$.

The reason is that the time $T_{\max}\Delta$ is where the second light cone intersects with the box boundary. If data beyond this limit is used, the wavefront will go outside of the box and the integrals would be underestimated. Thus, instead of positive infinity, numerically the upper bound for those integrals should be the x -coordinate that intersects with $t = T_{\max}\Delta$ at the 2nd light cone. Finally, we note that T_{step} should be wisely chosen so that after interpolation the output data can faithfully represent the result.

- [1] P. Lodahl, S. Mahmoodian, S. Stobbe, Rev. Mod. Phys. 87 (2015) 347–400. doi:[10.1103/RevModPhys.87.347](https://doi.org/10.1103/RevModPhys.87.347).
- [2] D. Roy, C. M. Wilson, O. Firstenberg, Rev. Mod. Phys. 89 (2017) 021001. URL: <https://link.aps.org/doi/10.1103/RevModPhys.89.021001>. doi:[10.1103/RevModPhys.89.021001](https://doi.org/10.1103/RevModPhys.89.021001).
- [3] C. Noh, D. G. Angelakis, Reports on Progress in Physics 80 (2017) 016401. URL: <http://stacks.iop.org/0034-4885/80/i=1/a=016401>. doi:[10.1088/0034-4885/80/1/016401](https://doi.org/10.1088/0034-4885/80/1/016401).
- [4] Z. Liao, X. Zeng, H. Nha, M. S. Zubairy, Physica Scripta 91 (2016) 063004. URL: <http://stacks.iop.org/1402-4896/91/i=6/a=063004>. doi:[10.1088/0031-8949/91/6/063004](https://doi.org/10.1088/0031-8949/91/6/063004).
- [5] X. Gu, A. F. Kockum, A. Miranowicz, Y.-X. Liu, F. Nori, Physics Reports 718 (2017) 1–102. doi:[10.1016/j.physrep.2017.10.002](https://doi.org/10.1016/j.physrep.2017.10.002).
- [6] H. Zheng, H. U. Baranger, Phys. Rev. Lett. 110 (2013) 113601. URL: <http://link.aps.org/doi/10.1103/PhysRevLett.110.113601>. doi:[10.1103/PhysRevLett.110.113601](https://doi.org/10.1103/PhysRevLett.110.113601).
- [7] T. Tufarelli, M. S. Kim, F. Ciccarello, Phys. Rev. A 90 (2014) 012113. doi:[10.1103/PhysRevA.90.012113](https://doi.org/10.1103/PhysRevA.90.012113).
- [8] Y.-L. L. Fang, H. U. Baranger, Phys. Rev. A 91 (2015) 053845. doi:[10.1103/PhysRevA.91.053845](https://doi.org/10.1103/PhysRevA.91.053845), *ibid.* **96**, 059904(E) (2017).
- [9] A. L. Grimsmo, Phys. Rev. Lett. 115 (2015) 060402. doi:[10.1103/PhysRevLett.115.060402](https://doi.org/10.1103/PhysRevLett.115.060402).
- [10] T. Ramos, B. Vermersch, P. Hauke, H. Pichler, P. Zoller, Phys. Rev. A 93 (2016) 062104. URL: <https://link.aps.org/doi/10.1103/PhysRevA.93.062104>. doi:[10.1103/PhysRevA.93.062104](https://doi.org/10.1103/PhysRevA.93.062104).
- [11] H. Pichler, P. Zoller, Phys. Rev. Lett. 116 (2016) 093601. doi:[10.1103/PhysRevLett.116.093601](https://doi.org/10.1103/PhysRevLett.116.093601).
- [12] P. Longo, P. Schmitteckert, K. Busch, Phys. Rev. Lett. 104 (2010) 023602. doi:[10.1103/PhysRevLett.104.023602](https://doi.org/10.1103/PhysRevLett.104.023602).
- [13] P. Longo, P. Schmitteckert, K. Busch, Phys. Rev. A 83 (2011) 063828. doi:[10.1103/PhysRevA.83.063828](https://doi.org/10.1103/PhysRevA.83.063828).
- [14] B. Peropadre, D. Zueco, D. Porras, J. J. García-Ripoll, Phys. Rev. Lett. 111 (2013) 243602. URL: <http://link.aps.org/doi/10.1103/PhysRevLett.111.243602>. doi:[10.1103/PhysRevLett.111.243602](https://doi.org/10.1103/PhysRevLett.111.243602).

- [15] E. Sanchez-Burillo, J. Garcia-Ripoll, L. Martin-Moreno, D. Zueco, *Faraday Discuss.* 178 (2015) 335–356. doi:[10.1039/C4FD00206G](https://doi.org/10.1039/C4FD00206G).
- [16] T. Shi, D. E. Chang, J. I. Cirac, *Phys. Rev. A* 92 (2015) 053834. doi:[10.1103/PhysRevA.92.053834](https://doi.org/10.1103/PhysRevA.92.053834).
- [17] Ş. E. Kocabaş, *Phys. Rev. A* 93 (2016) 033829. doi:[10.1103/PhysRevA.93.033829](https://doi.org/10.1103/PhysRevA.93.033829).
- [18] Ş. Ekin Kocabaş, *Optics Letters* 41 (2016) 2533. doi:[10.1364/OL.41.002533](https://doi.org/10.1364/OL.41.002533).
- [19] I. M. Mirza, J. C. Schotland, *Phys. Rev. A* 94 (2016) 012309. doi:[10.1103/PhysRevA.94.012309](https://doi.org/10.1103/PhysRevA.94.012309).
- [20] L. Guo, A. Grimsmo, A. F. Kockum, M. Pletyukhov, G. Johansson, *Phys. Rev. A* 95 (2017) 053821. doi:[10.1103/PhysRevA.95.053821](https://doi.org/10.1103/PhysRevA.95.053821).
- [21] S. J. Whalen, A. L. Grimsmo, H. J. Carmichael, *Quantum Science and Technology* 2 (2017) 044008. URL: <http://stacks.iop.org/2058-9565/2/i=4/a=044008>.
- [22] P.-O. Guimond, M. Pletyukhov, H. Pichler, P. Zoller, *Quantum Science and Technology* 2 (2017) 044012. URL: <http://stacks.iop.org/2058-9565/2/i=4/a=044012>.
- [23] Á. Rivas, S. F. Huelga, M. B. Plenio, *Rep. Prog. Phys.* 77 (2014) 094001. doi:[10.1088/0034-4885/77/9/094001](https://doi.org/10.1088/0034-4885/77/9/094001).
- [24] H.-P. Breuer, E.-M. Laine, J. Piilo, B. Vacchini, *Rev. Mod. Phys.* 88 (2016) 021002. doi:[10.1103/RevModPhys.88.021002](https://doi.org/10.1103/RevModPhys.88.021002).
- [25] I. de Vega, D. Alonso, *Rev. Mod. Phys.* 89 (2017) 015001. doi:[10.1103/RevModPhys.89.015001](https://doi.org/10.1103/RevModPhys.89.015001).
- [26] S. R. White, *Phys. Rev. Lett.* 69 (1992) 2863–2866. doi:[10.1103/PhysRevLett.69.2863](https://doi.org/10.1103/PhysRevLett.69.2863).
- [27] U. Schollwöck, *Rev. Mod. Phys.* 77 (2005) 259–315. doi:[10.1103/RevModPhys.77.259](https://doi.org/10.1103/RevModPhys.77.259).
- [28] U. Schollwöck, *Annals of Physics* 326 (2011) 96 – 192. URL: <http://www.sciencedirect.com/science/article/pii/S0003491610001752>. doi:[http://dx.doi.org/10.1016/j.aop.2010.09.012](https://doi.org/http://dx.doi.org/10.1016/j.aop.2010.09.012).
- [29] Y.-L. L. Fang, F. Ciccarello, H. U. Baranger, *New J. Phys.* 20 (2018) 043035. doi:[10.1088/1367-2630/aaba5d](https://doi.org/10.1088/1367-2630/aaba5d).
- [30] T. Tufarelli, F. Ciccarello, M. S. Kim, *Phys. Rev. A* 87 (2013) 013820. doi:[10.1103/PhysRevA.87.013820](https://doi.org/10.1103/PhysRevA.87.013820).
- [31] U. Dorner, P. Zoller, *Phys. Rev. A* 66 (2002) 023816. doi:[10.1103/PhysRevA.66.023816](https://doi.org/10.1103/PhysRevA.66.023816).
- [32] R. D. Driver, *Ordinary and delay differential equations*, volume 20 of *Applied Mathematical Sciences*, Springer-Verlag, New York, 1977.
- [33] B. Zubik-Kowal, *Scholarpedia* 3 (2008) 2851. doi:[10.4249/scholarpedia.2851](https://doi.org/10.4249/scholarpedia.2851).
- [34] A. Taflove, S. C. Hagness, *Computational Electrodynamics: the Finite-Difference Time-Domain Method*, third ed., Artech House, Norwood, MA, 2005.
- [35] J. B. Schneider, *Understanding the Finite-Difference Time-Domain Method*, 2010. URL: <http://www.eecs.wsu.edu/~schneidj/ufdtd/>, <http://www.eecs.wsu.edu/~schneidj/ufdtd/>.
- [36] W. H. Press, S. A. Teukolsky, W. T. Vetterling, B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, third ed., Cambridge University Press, 2007.
- [37] F. W. J. Olver, D. W. Lozier, R. F. Boisvert, C. W. Clark, *NIST Handbook of Mathematical Functions*, Cambridge University Press, 2010.
- [38] H. U. Baranger, G. Calajò, Y.-L. L. Fang, F. Ciccarello, Photon Capture into a Bound State in the Continuum of a Waveguide-Qubit System, 2018. URL: <http://meetings.aps.org/Meeting/MAR18/Session/E28.6>, APS March Meeting 2018; F. Ciccarello *et al.*, in preparation.
- [39] S. Lorenzo, F. Plastina, M. Paternostro, *Phys. Rev. A* 88 (2013) 020102. doi:[10.1103/PhysRevA.88.020102](https://doi.org/10.1103/PhysRevA.88.020102).
- [40] Y.-L. L. Fang, H. U. Baranger, *Phys. Rev. A* 96 (2017) 059904. doi:[10.1103/PhysRevA.96.059904](https://doi.org/10.1103/PhysRevA.96.059904).
- [41] M. V. Gustafsson, T. Aref, A. F. Kockum, M. K. Ekström, G. Johansson, P. Delsing, *Science* 346 (2014) 207. doi:[10.1126/science.1257219](https://doi.org/10.1126/science.1257219).
- [42] A. Frisk Kockum, P. Delsing, G. Johansson, *Phys. Rev. A* 90 (2014) 013837. doi:[10.1103/PhysRevA.90.013837](https://doi.org/10.1103/PhysRevA.90.013837).
- [43] J. D. Joannopoulos, S. G. Johnson, J. N. Winn, R. D. Meade, *Photonic Crystals: Molding the Flow of Light*, second ed., Princeton University Press, 2008.
- [44] A. Gil, D. Ruiz-Antolín, J. Segura, N. M. Temme, *ACM Trans. Math. Softw.* 43 (2016) 26:1–26:9. URL: [http://doi.acm.org/10.1145/2972951](https://doi.acm.org/10.1145/2972951). doi:[10.1145/2972951](https://doi.org/10.1145/2972951).